



DAY 2 • BLOCK 1 • FRX

Day 2

Convert the threat model into a review plan

Choose what to inspect, what to ask, and what evidence to collect.

Leave with a plan someone can follow

Start with the supplied Threat Dragon model or its PDF.

- Identify the route, component, and threat you will review.
- Turn the threat into a question about the code.
- Specify the expected behavior and evidence needed.
- Assign a reviewer, time limit, and review order.

Your output: extend the starter plan from 2 entries to 5.

Set the review scope first

Purpose: check whether FRX protects report access and handles untrusted inputs within the local case boundaries.

Target: a baseline review of the learner's **vulnerable** state. Record the supplied revision or snapshot date.

Inventory: login/session, report read/search, document intake, analytics import/fetch, public routes, errors, and local stores.

Select a first pass: name the paths you will inspect now; list deferred paths with a reason and follow-up owner.

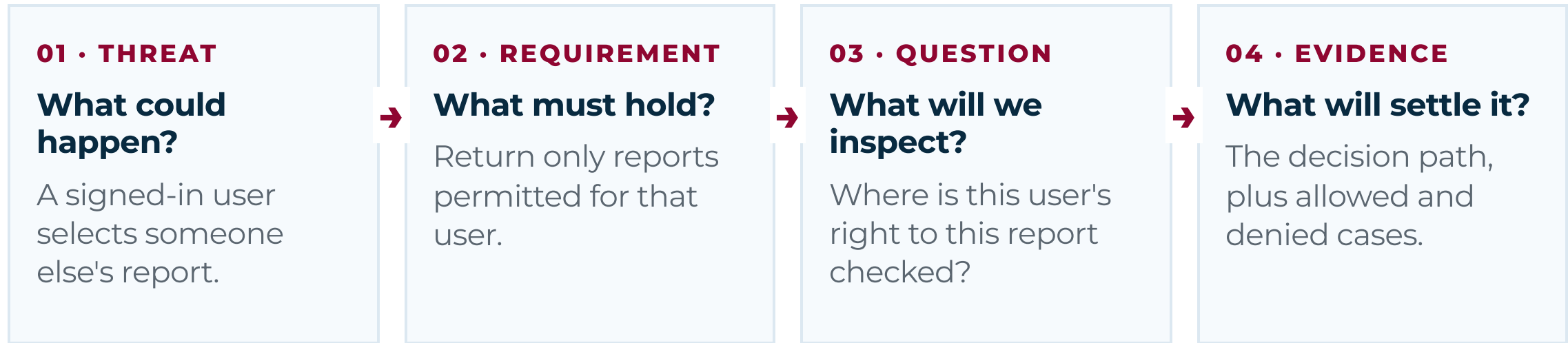
Read one threat all the way through

1. Open **diagram 06: FRX GET /reports/<int:report_id>**.
2. Select **FRX / Read report**.
3. Find **“Report identifier crosses ownership authority.”**
4. Read its precondition, asset, consequence, and proposed control.

Model status: Open. Treat the scenario as a question to investigate.

In the PDF: diagram on **page 43**, process threat on **page 50**.

Turn the scenario into review work



Keep the threat reference with the review entry.

Example 1: trace the report decision

Entry: `GET /reports/<int:report_id>`

Follow: session identity → chosen report → access decision → returned fields → read event or error.

Review question: can a valid session request a report outside the user's permitted set?

Expected rule: partners read their own reports; analysts read assigned reports; administrators read for authorized troubleshooting.

Start later at: `states/vulnerable.py:88–99`, then follow its helpers and database access.

Example 1: specify evidence before testing

Case to prepare	Expected behavior	Evidence to save later
Partner requests own report	Report is returned	Identity, report ownership, returned fields
Same partner requests unrelated report	No protected report data	Response and the authorization decision path
No session requests a report	No protected report data	Response and the authentication path
Analyst: assigned, then unrelated report	Only assigned report is returned	Assignment data and both results

Add administrator and missing-report cases to complete this entry.
Record **actual code lines and actual results** during the review.

Expected behavior derives from REQ-01 and ARC-02. Agree exact denial responses if the supplied contract does not define them.

Starter entry RP-01

What to review

Reference: diagram 06, Read report, “Report identifier crosses ownership authority.”

Question: is report authority checked before protected data is returned?

Start: `report_detail` and its identity/database helpers.

How to finish the entry

Evidence: decision-path lines; role/report cases; responses without secrets.

Order: early, because a failure could disclose reports.

Owner / time: assign a name / 20 minutes of later review.

Status: planned; not tested.

Example 2: a permitted host is one check

Diagram 10: GET /analytics/fetch → FRX / Fetch analytics

Threat: **“Provider path selection exceeds intended report authority.”**

- A signed-in caller selects a path on the configured provider origin.
- The model limits outbound requests to that origin.
- Review which paths and report identifiers the caller may request.
- The provider returns synthetic metrics; it does not read FRX reports.

Ask separately: “Is this destination allowed?” and “May this user request these metrics?”

Starter entry RP-02

Question: what restricts provider paths and report identifiers after the destination-origin check?

Start: `analytics_fetch`, URL parsing, configured origin, and provider routes.

Prepare cases: intended metric; unrelated report metric; another path on the same origin; a different origin that must be rejected before a request is sent.

Collect later: control-path lines, allowed-path policy, and observed request/response evidence. Confirm the intended analytics authority rule with the requirement owner.

Owner / time / status: assign a name / 15 minutes / planned.

Choose the order and reserve time

Consider	Write a concrete reason
Impact	“A failure could expose another user's report.”
Preconditions	“The caller needs only an ordinary session.”
Shared decisions	“Check login/session authority before relying on role tests.”
Uncertainty	“The allowed analytics paths need confirmation.”

Use the model's priority as a starting point. Explain your order.

Planning budget: 90 minutes of later review. RP-01 and RP-02 use 35; allocate the remaining 55 across your three new entries.

Choose people, tools, and references

People

Reviewer: reads the Python/Flask path and records evidence.

Partner: checks the trace and prepares cases.

Requirement owner: resolves policy gaps. Record a role if no person is assigned.

Materials

Now: model/PDF, planning basis, starter plan, checklist.

Later: editor/search; local HTTP client or focused tests for selected questions.

Choose a tool because it can produce the evidence your entry asks for.

Link the plan to the checklist

For RP-01, use the supplied checklist's **PDF page 3**:

“Authorization works properly and cannot be circumvented by parameter manipulation”

- Record the page and row wording in the plan.
- Specify the code and request evidence needed to assess that row.
- Leave **Pass / Fail / N/A** unmarked until the review supports a disposition.
- Keep unanswered questions in the plan with an owner.

Later, carry confirmed evidence and remaining gaps into APPROVE's **Scope** and **Risk Assessment Results**.

Hands-on: extend the plan • 20 minutes

Work in pairs. Open **review-plan-starter.md** and your model or PDF.

1. **3 min:** set scope, revision, names, and first-pass review order.
2. **10 min:** add **RP-03, RP-04, RP-05**, each from an attached threat on a different endpoint not covered by the starter.
3. **4 min:** specify allowed/error cases, evidence, a checklist link, owner, and review time for each.
4. **3 min:** exchange plans and revise one unclear entry.

Choose at least one of **login/session**, **search**, or **document intake**. Select the other two endpoints yourself.

Submit 5 entries, a review order, and a deferred-work list.

Keep the exercise inside the case

- Plan the inspection and cases now; execute them during the code crawl.
- Intake stores standalone files. No report attachment or retrieval workflow is supplied.
- Import inspects serialized input without execution or persistence.
- Fetch selects paths within the configured local provider origin.
- Mark host permissions, key custody, recovery, and capacity evidence as **unverified** unless supplied.

If a policy is missing: write the question, identify who should answer, and state what review work can still proceed.

Check another pair's plan

- Can you locate each threat by diagram, component, and title?
- Does each question identify a specific decision or data flow?
- Are expected behavior and observed results kept separate?
- Could you collect the requested evidence without guessing?
- Do the owners, order, and time limits fit the 90-minute budget?
- Are deferred paths and unanswered policy questions visible?

Replace “check input validation” with a named input, rule, case, and evidence request.

Brief one entry, then start the crawl

In one minute, explain:

1. **The threat:** who can do what, to which asset?
2. **The review question:** what decision or flow will you inspect?
3. **The proof:** what code evidence and cases will answer it?
4. **The order:** why spend review time here now?

Save your plan. During the next block, add file/line evidence, observed behavior, checklist disposition, and unresolved questions to these same entries.

DAY 2 • BLOCK 2

Review strategy, Code Crawl, and JWAC checklist

Learn a method for inspecting code.

Use a reviewer questionnaire to check broader security coverage.

Pick the review type

Baseline review	Change review
Establish coverage across an application's current code and configuration.	Start with a named change and follow its effects.
FRX: inventory routes, shared helpers, stores, and dependencies.	Example: a change to <code>_authenticated</code> requires revisiting every caller.
Use the threat model to order the work; track paths still unreviewed.	Read the diff, surrounding code, callers, and affected trust boundaries.

Today: a baseline review, performed in selected slices. Your five plan entries are the first pass.

What OWASP means by crawling code

OWASP's **Code Review Guide v2**, §6.11, calls this “**Crawling Code.**”

- Search for entry points and APIs that handle input, identity, files, databases, and output.
- Open the matches and follow the surrounding logic and helper calls.
- Examine business rules even when no dangerous API name appears.

The guide's appendix provides language-specific search lists. The next slides adapt the technique to **Python/Flask in FRX**.

A search match is a place to inspect. It is not a finding.

Orient yourself before searching

Read	Locate
<code>app.py:12–28</code>	Configuration, selected state, blueprint registration, browser root
<code>states/vulnerable.py</code>	Learner routes, identity helper, audit helper
<code>database.py</code> / <code>config.py</code>	Data fields, database access, file locations, request limit
<code>templates/</code> / <code>static/client.js</code>	Browser requests and display of returned values
<code>fixture_service.py</code>	Separate analytics-provider routes
<code>frx-project/pyproject.toml</code>	Direct dependencies and entry points

Record the state and revision in your evidence notes before tracing.

First sweep: find the routes

Run from `frx-project/src/frx_portal`, or use your editor's file search.

```
rg -n '@(app|blueprint)\.(get|post|route)' app.py states/vulnerable.py  
rg -n '@app\.(get|post|route)' fixture_service.py
```

- Match each result to **service + HTTP method + route** in the model.
- Check public endpoints as well as routes used by the browser UI.
- Add framework static routes and automatic methods from the inventory; they will not all appear as decorators.

Record gaps: an entry point absent from the model, or a modeled route you cannot locate.

Second sweep: find inputs and operations

Run against the learner state, then follow its imports and helpers.

```
rg -n 'request\.|session|_authenticated' states/vulnerable.py
rg -n 'execute|\.save\(|urlopen|genops|_audit|jsonify' states/vulnerable.py
```

Match

Next question

`request.args` , `request.files`

Who controls the value? What checks apply?

`execute` , `save` , `urlopen`

What reaches this operation, under whose authority?

`_audit` , `jsonify`

What leaves the process or enters a log?

Trace in both directions

Forward from an input

Choose a value. Follow its checks, conversions, helper calls, and uses.

FRX: `name` in `GET /welcome` reaches an HTML response.

Backward from an operation

Choose a sensitive operation. Find every input and control that can reach it.

FRX: `uploaded.save(destination)` leads back to the submitted filename and intake directory.

At each step, record the value, who controls it, the applicable check, and the source line.

Worked crawl: a public HTML response

Route: GET /welcome · **State:** vulnerable · **Model:** diagram 05

```
@blueprint.get("/welcome")
def welcome():
    return f"<h1>Welcome, {request.args.get('name', 'researcher')}</h1>"
```

Source: the caller supplies **name** in the query string.

Processing: the value is inserted directly into a string; no HTML escaping appears on this path.

Output: Flask treats the returned string as HTML. Trace the response context as well as the input.

frx-project/src/frx_portal/states/vulnerable.py:83–85 (exact excerpt). Flask Quickstart — HTML Escaping. Static source observation; no request executed for this example.

Check each output context

Output path	What the code shows	Limit of the conclusion
<code>/welcome</code> response	Query value inserted into HTML, <code>vulnerable.py:85</code>	Assess this direct route even if the main UI never calls it.
Report titles in the browser	<code>title.textContent = report.title</code> , <code>client.js:65</code>	This assignment displays text; it does not validate every output in FRX.

Next step: list other response and rendering locations, then inspect each relevant context.

Keep the conclusion limited to the output you inspected.

Follow branches and side effects

For `POST /documents`, read beyond the file-save call.

Branch or step	Where to look	Review question
Missing session	<code>vulnerable.py:115–116</code> , helper 42–45	Does processing stop before intake?
Empty client name	<code>vulnerable.py:120–122</code>	What response is returned?
Save → audit → response	<code>vulnerable.py:123–127</code> , helper 37–39	If the audit write fails, what has already changed?
Unhandled failure	<code>app.py:34–39</code>	What can the caller see?

Record **operation order**. An error response alone does not tell you whether a file was written.

The checklist questionnaire

Use [Code_Review_Checklist_Fillable.pdf](#) as a set of general security questions. It helps you identify topics to consider and questions to ask about an application.

Page	Examples of topics on that page
1	Authorization placement, stopped execution, business logic
2	Input validation, transport, logs, graceful failure
3	Keys, libraries, role rules, parameter/cookie manipulation
4	Server-side validation, output encoding, XML/WSDL, boundaries

Use it to check the breadth of your review and identify unanswered questions. The questions cover design, implementation, configuration, and operating practices.

Use the questions throughout a review

Before the review: scan the topics. Identify relevant features, people to consult, and documents or access you will need.

During the review: use the questions to challenge assumptions and notice subjects that have not been examined.

At the end: revisit unanswered questions. Record what is supported, what needs clarification, and who will follow up.

Answers may come from design documents, configuration, source review, tests, or discussions with the people who operate the system.

Answer with a reason

Response	What to explain
Pass	Why the security expectation is met, and what information supports that answer.
Fail	What falls short of the expectation and what needs attention.
N/A	Why the question does not apply to this application or review scope.
Unanswered in notes	What is still unknown, who can answer, and what information is needed.

Read the wording carefully: **Yes does not always mean Pass.** Leave a question unmarked when you lack enough information to answer it.

Example question: are privileges documented?

PDF p. 3: “User and role based privileges are documented”

Ask for: a description of each role, the actions and data it may access, and who maintains that description.

FRX example: REQ-01 describes intended report access for partners, analysts, and administrators. That is useful documentation to examine.

Follow-up: does it cover all relevant functions? Who confirms the intended permissions and keeps them current?

A documented access rule establishes intent. Whether the application enforces it is a separate review question.

Example question: who reviews the logs?

The questionnaire asks

“Is there a process(s) in place to read audit logs for unintended/malicious behaviors?”

Purpose: establish whether recorded events receive attention and lead to action.

Ask the operating team

Who reviews the logs, and when?

What triggers investigation or escalation?

What procedure and completed review records support the answer?

Record unanswered points and a follow-up owner.

Read the wording before marking a box

PDF p. 2 asks: “Are logs logging personal information, passwords or other sensitive information?”

A **Yes** to that question may be evidence of a security failure. Pass/Fail must reflect the agreed security criterion, not whether the answer is Yes/No.

Criterion to assess: credentials and session secrets must not appear in application logs.

Ask for: the logging policy, configured event fields, and appropriately redacted sample records. Explain any sensitive data present and the handling rules that apply.

Resolve wording conflicts explicitly

Supplied wording	What to record and clarify
“Is password stored in an encrypted format?” — p. 1	Distinguish password hashing from reversible encryption; record algorithm and work factor. Confirm the accepted interpretation.
“Session cookies are encrypted” — p. 3	Flask's default session is signed and readable. Record integrity, confidentiality, contents, and transport separately.
“Does the centralized validation check block all the special characters?” — p. 2	Identify valid input and output context. A blanket character ban is not a complete injection control.
Preserve the original wording. Record the conflict and proposed interpretation; refer unresolved policy decisions to the checklist owner.	

Separate N/A from missing evidence

N/A needs a scope reason

Candidate: the XML-schema row on p. 4.

Reconcile the interfaces, body parsers, and dependencies. If no XML input is accepted in the reviewed scope, document that evidence.

A search with no matches is only one check.

Unknown needs an owner

Example: the log-monitoring-process row on p. 2.

Finding an audit writer does not prove anyone monitors it.

Request the procedure, responsible role, and operating evidence. Keep the row pending until assessed.

Pair practice: a small crawl • 10 minutes

Use `POST /logout`, model diagram 04, and **code-crawl-worksheet.md**.

1. **2 min:** locate the route and every helper it calls.
2. **3 min:** follow session input → event write → session change → response. Include an event-write failure branch.
3. **3 min:** record two code observations or unanswered technical questions, with source lines and the next check needed.
4. **2 min:** exchange records. Challenge any conclusion that goes beyond the evidence.

Submit one request trace and two review notes.

Read-only exercise in the vulnerable state; do not execute requests or change code in this block. Start at `frx-project/src/frx_portal/states/vulnerable.py`.

Carry the record into the full review

- A route is covered only when its relevant branches, outputs, and side effects have evidence or explicit gaps.
- Shared helpers can supply evidence to several entries; record which callers were checked.
- Record which code and behavior you inspected. Keep deferred work visible.
- Separate a code observation, a tested result, and an unresolved policy question.

Next: use the Python security-surface block to recognize more inputs and operations. Then apply this process to your five plan entries during the manual crawl.

References for block 2

- [OWASP Code Review Guide v2](#): §6.11, printed p. 42 / PDF p. 43; Code Crawling appendix, printed pp. 205–217. Python search examples are course adaptations.
- [OWASP Secure Code Review Cheat Sheet](#): review types, review process, and data-flow analysis.
- **Reviewer questionnaire:** [Code_Review_Checklist_Fillable.pdf](#), PDF pp. 1–4 / printed pp. 196–199. General questions for checking the breadth of a security review.
- **FRX evidence:** learner-state source, threat model, and [models/requirements-and-architecture.md](#). Code paths and line references appear with each example.

Reading the Python security surface

DAY 2 • BLOCK 3 • INSTRUCTOR COMPARISONS

Read the input, the operation, and the control.
Explain what changed—and what still needs review.

Read the Python behavior, not the label

Compare the same operation in **vulnerable.py** and **remediated.py**.

- Identify the caller-controlled value and its runtime type.
- Find the operation: query, template, file write, parser, log, or request.
- Read the check and every return/exception path around it.
- Explain the correction, then identify what it does not establish.

“Remediated” describes selected corrections. Every route still needs review.

Code excerpts show selected lines, not complete handlers. Proposed and non-FRX examples are labeled.

Small Python constructs carry review meaning

Construct you see	What to check
<code>def report_detail(report_id: int)</code>	The annotation documents intent. Find the runtime conversion/check.
<code><int:report_id></code> in a Flask route	The router converts the value; it does not grant report access.
<code>if response := _authenticated():</code>	The helper result is assigned and tested. Read what it returns.
<code>with connect(...) as connection:</code>	Read the context manager's commit, rollback, and cleanup behavior.
<code>f'...{value}...'</code>	Ask whether the result becomes SQL, HTML, a filename, or plain data.

JSON parsing does not guarantee a dictionary

FRX vulnerable login

```
payload = request.get_json(silent=True) or  
request.form  
username = payload.get("username", "")  
password = payload.get("password", "")
```

Reads fields without checking the top-level JSON type.

FRX remediated login

```
payload = request.get_json(silent=True) or  
request.form  
username = str(payload.get("username",  
"")).strip()  
password = str(payload.get("password", ""))
```

Converts field values to strings; this still assumes `payload` has a `get` method.

Review case: a nonempty JSON array reaches `.get()` in both versions. A number used as a password becomes a string in the remediated code.

Proposed check: validate the object shape and field types before coercion; decide explicitly whether both JSON and form inputs are supported.

SQL: separate the statement from values

Vulnerable: value inside SQL

```
query_text = request.args.get("q", "")
query = f"SELECT * FROM reports WHERE title LIKE
'{query_text}%' OR summary LIKE '%
{query_text}%'
with connect(_settings().database_path) as
connection:
    reports = [dict(row) for row in
connection.execute(query).fetchall()]
```

Caller text becomes part of the SQL statement.

Review: parameter binding protects query structure. Check report permissions and result size separately. `%` and `_` still have LIKE-pattern meaning.

Remediated: values passed separately

```
rows = connection.execute(
    "SELECT * FROM reports WHERE title LIKE ? OR
summary LIKE ?",
    (f"%{query_text}%", f"%{query_text}%"),
).fetchall()
```

The SQL contains placeholders. Even the wildcard f-strings are passed as parameter values.

Passwords: read storage and verification together

FRX vulnerable hashing

```
def password_hash(password: str) -> str:  
    return hashlib.md5(password.encode("utf-  
8")).hexdigest() # noqa: S324 - deliberate lab  
flaw
```

Uses a fast MD5 digest; no per-password salt is added here.

Also compare login: the remediated route checks `active`, clears prior session data, and returns selected account fields.

Still review: guessing limits, password policy, recovery, and the cost settings of the installed library. These are not established by a hash call.

FRX remediated hashing

```
def password_hash(password: str) -> str:  
    return generate_password_hash(password,  
method="scrypt")
```

Uses Werkzeug's salted scrypt password hash. Login verifies it with `check_password_hash`.

Sessions: flags, keys, and identity lifetime

Vulnerable cookie configuration

```
SESSION_COOKIE_SECURE=False,  
SESSION_COOKIE_HTTPONLY=True,  
SESSION_COOKIE_SAMESITE=None,
```

A fixed training signing key is configured in source.

Flask's default cookie session is **signed, not encrypted**. Secure requires HTTPS; HttpOnly restricts script access; SameSite limits some cross-site sending.

Still review: key-file permissions, rotation, cookie lifetime, replay after logout, and stale role/account data. SameSite alone does not settle CSRF protection.

Remediated cookie configuration

```
SESSION_COOKIE_SECURE=True,  
SESSION_COOKIE_HTTPONLY=True,  
SESSION_COOKIE_SAMESITE="Lax",
```

The signing key comes from the environment or a generated local key file.

A session check is not report authorization

Authentication helper in both states

```
def _authenticated():  
    if "user_id" not in session:  
        return jsonify(error="Login required"),  
401  
    return None
```

Checks only whether `user_id` exists in the session.

Additional remediated policy

```
return bool(  
    role == "administrator"  
    or (role == "partner" and report["owner_id"]  
    == user_id)  
    or (role == "analyst" and  
    report["assigned_analyst_id"] == user_id)  
)
```

Checks the role plus owner/assignment fields for this report.

Vulnerable report reads stop at authentication; remediated reads call `_can_read`. **Still review:** administrator troubleshooting restrictions and session role revalidation.

Check every route that can return the data

Vulnerable search results

```
reports = [dict(row) for row in  
connection.execute(query).fetchall()]  
return jsonify(reports=reports, query=query)
```

Returns all matching rows and the query text.

Remediated search results

```
reports = [dict(row) for row in rows if  
_can_read(row)]  
return jsonify(reports=reports)
```

Applies the report policy to each returned row and omits query text.

A protected detail route does not protect a separate search route automatically.

Still review: `fetchall()` retrieves all matches before filtering. There is no pagination/result cap here. Search is not audited in either state.

HTML: keep template source separate from data

Vulnerable greeting

```
return f"<h1>Welcome, {request.args.get('name',  
'researcher')}</h1>"
```

Caller text is inserted directly into HTML.

Remediated greeting

```
return render_template_string(  
    "<h1>Welcome, {{ name }}</h1>",  
    name=request.args.get("name", "researcher")  
)
```

The template is fixed; **name** is a value subject to Flask/Jinja autoescaping.

Review: inspect the output context and any **Markup**, **|safe**, or disabled-autoescape use. HTML escaping is not a JavaScript or URL policy.

Beyond FRX: passing user-controlled template text to **render_template_string** lets the caller supply template instructions; autoescaping does not make that safe.

Files: a safe basename can still overwrite data

Vulnerable intake naming

```
client_name =  
PurePosixPath(uploaded.filename.replace("\\",  
"/")).name
```

Uses the submitted basename. Reusing a name can select an existing intake file.

Remediated intake naming

```
stored_name = f"{uuid.uuid4().hex}{suffix}"  
destination = (_settings().upload_root /  
stored_name).resolve()  
upload_root = _settings().upload_root.resolve()
```

Generates a storage name, resolves the path, and checks its containment.

Both states keep ordinary upload writes inside disposable intake storage. This case does not demonstrate an arbitrary-path write.

Review: naming collisions, overwrite behavior, filesystem permissions, and what is acknowledged. There is no report association or retrieval workflow.

File checks have a defined strength

Vulnerable: accepts the named file and saves it without content/type checks.

Remediated: permits `.json`, `.pdf`, `.txt`; checks the content before saving.

```
data = uploaded.stream.read(_settings().max_upload_bytes + 1)
uploaded.stream.seek(0)
if not data or len(data) > _settings().max_upload_bytes:
    return False
if suffix == ".pdf":
    return data.startswith(b"%PDF-")
```

The helper rewinds the stream after reading. Its PDF check only recognizes a header; that does not establish that a document is harmless.

Review: per-request bytes, parser work, aggregate storage, and failure handling. Both states set a request-size limit; neither demonstrates a storage quota.

Serialized input: identify what actually executes

Vulnerable FRX import: inspection

```
decoded = base64.b64decode(request.get_data(),  
                             validate=True)  
opcodes = [opcode.name for opcode, _argument,  
            _position in pickletools.genops(decoded)]
```

Decodes base64 and inspects pickle opcodes with `pickletools.genops`. It does not unpickle or execute them.

Remediated FRX import: JSON

```
if not isinstance(value, dict) or set(value) -  
{"report_id", "score", "category"}:  
    return jsonify(error="Unexpected analytics  
fields"), 400
```

Restricts the object's fields and checks selected value types. Parsing and schema rules still need separate review.

Beyond FRX: `pickle.loads(untrusted_bytes)` can execute code. Do not substitute it into this demonstration.

Both FRX import routes return an assessment/result without persistence or a report-ownership decision.

Python numeric checks can accept booleans

The remediated import uses `isinstance(..., int)` and `isinstance(..., (int, float))`.

```
isinstance(True, int)           # True
isinstance(False, (int, float)) # True
isinstance(float("nan"), (int, float)) # True
```

Review cases: boolean IDs/scores, negative IDs, nonfinite scores, missing `category`, and unsupported category values.

Proposed rules: reject booleans explicitly; require finite numbers; specify ranges and required fields. Agree the score/category contract before asserting a failure against it.

Catch the exception the framework raises

Remediated import excerpt:

```
try:
    value = request.get_json(force=False)
except (TypeError, json.JSONDecodeError):
    return jsonify(error="Invalid analytics document"), 400
```

Flask normally raises **BadRequest** for malformed JSON with a JSON content type. That is not **json.JSONDecodeError**; the custom message above is not guaranteed on that path.

Review: follow the framework exception and error handler. The earlier MIME check already returns 415 for a non-JSON content type.

Proposed correction: handle the intended framework exception or use an agreed central error handler; verify status, body, and content type.

Outbound calls: destination and authority differ

Vulnerable: caller selects a URL

```
destination = request.args.get("url", "")
allowed =
    urlsplit(_settings().analytics_base_url)
    requested = urlsplit(destination)
```

The next checks restrict scheme, host, port, and user information to the configured fixture origin.

Actual contrast: caller-selected paths on the permitted origin versus a report-authorized metrics operation.

Review: URL parsing, redirects, timeouts, and who controls the provider. The unchanged local fixture has no redirect workflow; unrestricted outbound access is not demonstrated.

Remediated: portal constructs a path

```
destination = f"
{_settings().analytics_base_url}/metrics/{report
_id}"
```

Before this excerpt, the route validates the ID, loads the report, and checks `_can_read`.

Provider responses are still untrusted input

Vulnerable response handling

```
with urllib.request.urlopen(destination,  
    timeout=2) as remote: # noqa: S310  
    payload = remote.read(64 *  
        1024).decode("utf-8")  
    return current_app.response_class(payload,  
        mimetype="application/json")
```

Reads limited bytes, decodes them, and labels the result as JSON without parsing it.

Still review: score/category semantics, invalid byte encoding, and exact type checks. Python equality treats `True == 1` as true.

A 64 KiB read is a read limit, not explicit oversized-response rejection. Check a limit-plus-one strategy and a complete timeout/resource budget if required.

Remediated identity check

```
if not isinstance(decoded, dict) or  
    decoded.get("report_id") != report_id:  
    return jsonify(error="Analytics response  
        failed validation"), 502
```

Parses JSON and checks the returned report ID before responding.

Logging: named fields do not escape their values

Vulnerable login audit call

```
_audit(f"LOGIN ATTEMPT username={username}  
password={password}")
```

Includes the supplied password in the audit message.

The remediated `_audit` joins `key=value` strings and writes a newline. It does not escape embedded newlines in values; the name `safe_fields` is not evidence of encoding.

Review: redaction, event boundaries, injection into log records, coverage, retention, and write failures. Consider a structured serializer with an explicit field policy.

Remediated login audit call

```
_audit("login", "failed", username=username)
```

Omits the password, but still includes the supplied username on this failure path.

Ordering and cleanup are security behavior

Vulnerable logout sequence

```
_audit(f"LOGOUT username={session.get('username')}")  
session.clear()  
return jsonify(status="logged-out")
```

A failed audit write occurs before the session is cleared.

Remediated logout sequence

```
username = session.get("username", "unknown")  
session.clear()  
_audit("logout", "success", username=username)  
return jsonify(status="logged-out")
```

Clears the session before the audit write; the audit can still raise an exception.

Review: what has changed before an exception? What response reaches the client, and does it update the cookie as intended? Verify the whole failure path.

A **with** block can close a file or roll back a database transaction. It does not make file, log, database, and cookie changes one atomic operation.

Errors: inspect the returned fields and handlers

Vulnerable shared error handler

```
if state == "vulnerable":  
    return jsonify(error=str(error),  
                  traceback=traceback.format_exc()), 500
```

Returns exception details and a traceback to the caller.

Remediated shared error handler

```
app.logger.exception("Unhandled FRX request  
failure")  
return jsonify(error="The request could not be  
completed."), 500
```

Logs the exception and returns a generic response to the caller.

The dedicated diagnostic route also differs: vulnerable code returns an internal path and signing secret; remediated code returns a generic error and records an event.

Review: every error path and logger destination. A generic response does not prove that operator logs are free of secrets or access-controlled.

Review configuration and dependencies

Python review surface	FRX example and question
Environment and paths	<code>FRX_SESSION_SECRET</code> , runtime root, provider URL: who controls them?
Process privileges	Files are opened under the process account. What else can that account read or write?
Shared limits	<code>MAX_CONTENT_LENGTH</code> comes from settings. Which other work needs a limit?
Imports and packages	Review direct dependencies plus transitive Flask/Jinja/Werkzeug behavior.
Launch behavior	The selected blueprint and HTTPS settings change with the chosen state.

Compare declared packages with the installed environment. A pin alone establishes neither secure configuration nor vulnerability status.

config.py:15–61; app.py:12–24; cli.py:9–25; states/remediated.py:23–34; frx-project/pyproject.toml; requirements.txt. No dependency vulnerability assessment was performed.

Beyond FRX: avoid turning input into shell syntax

Risky pattern — illustrative

```
subprocess.run(  
    f"tool {name}", shell=True  
)
```

The shell interprets the constructed command string.

Safer construction — illustrative

```
subprocess.run(  
    [tool, "--", name],  
    shell=False, timeout=5, check=True  
)
```

Arguments are separate. Here **tool** must be a fixed trusted absolute executable path and support **--**.

Review: validate the operation and arguments, process privileges, environment, output volume, and timeout handling. Removing the shell does not authorize the operation or constrain the program's own argument language.

Beyond FRX: do not enforce access with assert

Risky authorization check

```
assert user.is_admin  
perform_admin_action()
```

Python optimization can remove the assertion entirely.

Explicit runtime decision

```
if not user.is_admin:  
    raise PermissionError("Access denied")  
perform_admin_action()
```

The conditional and rejection remain in optimized execution.

Review: assertions are for developer assumptions, not required checks on untrusted input or user authority. Map the rejection to the application's intended error response.

Also establish where `user.is_admin` comes from and whether it is current.

Beyond FRX: mutable defaults can share user data

One list reused across calls

```
def remember(report_id, seen=[]):  
    seen.append(report_id)  
    return seen
```

The default list is created once. Later calls can observe earlier values.

A new list for each defaulted call

```
def remember(report_id, seen=None):  
    if seen is None:  
        seen = []  
    seen.append(report_id)  
    return seen
```

Avoids accidental default-state sharing; supplied lists can still be intentionally shared.

Review: mutable defaults, class attributes, module globals, and caches that hold user-specific data. Check identity keys and lifetime before using them in a request handler.

Explain a correction and its limit • 5 minutes

Choose one FRX comparison from this block.

1. Name the route, input, and sensitive operation.
2. Show the vulnerable and remediated lines that change the behavior.
3. Explain exactly which threat the correction addresses.
4. Identify one remaining question or unhandled case.
5. Propose an intended-use case and an edge/error case to verify later.

Submit a short explanation with source lines. Distinguish observed implementation, proposed controls, and results you have not tested.

Python references for this block

- **FRX source:** `states/vulnerable.py` , `states/remediated.py` , shared application/configuration/database code. Exact excerpt locations are on each slide.
- **Python 3.13:** [type hints](#), [sqlite3](#), [pickle](#), [JSON](#), [urllib](#), [subprocess](#).
- **Flask:** [request API](#), [templates](#), [security considerations](#).
- **Werkzeug:** [password hashing and utilities](#).

Examples describe the supplied source. Remediation comparisons do not establish production readiness or a completed security assessment.

DAY 2 • PRACTICUM 3

Review by risk and governed criteria

Use the Python examples and your review notes to answer selected checklist questions about FRX.

Produce evidence, supported dispositions, and specific follow-up work.

Choose the review order by consequence

Pick your first two questions from the review plan. Explain the order.

- **Reachability:** public caller, signed-in partner, or local file access?
- **Consequence:** disclosure of reports, identity misuse, file loss, or unreliable audit evidence?
- **Limits:** existing checks, loopback, synthetic data, or no retrieval route?
- **Uncertainty:** which deployment or operating assumptions need evidence?

Record deferred questions and their owners. Ease of discovery does not establish priority.

Make each checklist answer reviewable

Record	What to write
Question and scope	PDF page + exact wording; route/state/revision inspected
Applicability and rule	Why it applies; expected behavior and its source
Evidence	File:lines, relevant branch, and what the evidence establishes
Disposition	Proposed Pass / Fail / N/A, with a reason and coverage limits
Follow-up	Finding or open-question ID; next evidence; owner and due point

Insufficient evidence: leave the PDF boxes unmarked and track the question in your notes. A negative answer can mean a safe result when the question asks about a dangerous behavior.

Worked record: one output, one conclusion

Question: “Has the correct encoding been applied to all data being output by the application” — PDF p. 4, printed p. 199.

Field	Instructor example: vulnerable GET /welcome
Expected behavior	Caller text remains data in the HTML response.
Source evidence	<code>states/vulnerable.py:83–85</code> puts <code>name</code> directly in returned HTML.
Proposed disposition	Fail: this output is a counterexample to the whole-app claim.
Coverage limit	Static observation; runtime validation and other outputs remain open.
Next step	Reviewer checks harmless markup in later validation and records the response.

One counterexample supports failure of an “all outputs” claim. One safe output cannot establish a pass for all outputs.

Full source root: `frx-project/src/frx_portal/`. Framework basis and worked trace: slides 25–26; this is a teaching record, not a completed questionnaire.

A • Inputs, output, and failure paths

Checklist: “Are all the untrusted inputs validated?” (PDF p. 2); “Is the program designed to fail gracefully?” (PDF p. 2).

Choose `/documents` or `/analytics/import` and inspect:

1. The input format, allowed values, and size checks that actually run.
2. The rejected-input and exception branches; any earlier file or log write.
3. The returned status and data. What would a caller learn?

Write **one ordinary case and one boundary/error case**, with expected behavior. Distinguish the shared request-size limit from route-specific checks.

B • Access and session decisions

Checklist: “Authorization works properly and cannot be circumvented by parameter manipulation”; “Session data is validated” (PDF p. 3).

- For `/reports/<int:report_id>`, compare an own-report request with another partner's report. Where is that distinction enforced?
- Inspect `/search` separately. Can the same report data leave through this path?
- Follow the session values back to `/login`. What is trusted later, and what is checked again?

Record **one access decision and one session question**. Cite the rule as well as the implementation; report existence and identity alone do not establish permission.

C • Secrets and transport

Checklist: “Keys are not held in code.” (PDF p. 3); “Is all PI and sensitive information being sent over the network encrypted form.” (PDF p. 2).

1. Locate the session-signing key and every response that could disclose it.
2. Check browser-to-portal and portal-to-provider transport separately.
3. Identify what source configuration establishes and what needs host or operational evidence.

Record the **local lab behavior**. Loopback and synthetic data constrain this exercise; they do not establish an approved exception for a deployed system.

D • Audit events and operating practice

Checklist: “Do audit logs log connection attempts (both successful and failures)?”; “Is there a process(s) in place to read audit logs for unintended/malicious behaviors?” (PDF p. 2).

- Trace valid login, invalid credentials, and an error branch. Which events and fields can each path write?
- Check whether a log reader can distinguish the result without exposing credentials.
- Identify the evidence needed for log access, retention, review frequency, and response ownership.

Keep **event generation** and **human/operational review** as separate answers. A call to `_audit` establishes neither complete coverage nor a monitoring process.

Checklist PDF p. 2 / printed p. 197. Start: states/vulnerable.py:37–39, 53–80; config.py:44–46; model REQ-05 / ASM-01. Inspect source; do not collect runtime passwords or secrets.

E • Build and dependency evidence

Checklist anchor: the external-library question on PDF p. 3 covers versions, patching, and the update process.

Inspect or request	What it helps establish
<code>pyproject.toml</code> , <code>requirements.txt</code>	Declared packages and version constraints
Environment inventory / SBOM	Components present in the assessed build
Dated analysis + triage record	Which tool claims were checked, by whom, and against which build
Review, update, and exception records	Ownership, thresholds, follow-up, and required decisions

Identify supplied and requested evidence. **A pin is not a vulnerability assessment.** Practicum 4 validates tool, SBOM, and license evidence.

Checklist PDF p. 3 / printed p. 198, General external-library question. Course outline adds process/supply-chain work; SBOM, license, and review-record requests are not invented PDF rows.

Add precise criteria to the evidence

Use the questionnaire to frame the question. Add a testable criterion and a weakness label when they help explain the observation.

Observed behavior	Verification criterion	Weakness / risk category
<code>/search</code> places caller text in SQL	ASVS v5.0.0-1.2.4: protect database queries from injection	CWE-89; OWASP A05:2025 Injection
Report read lacks a report-permission decision	ASVS v5.0.0-8.2.2: restrict access to permitted data items	CWE-862; OWASP A01:2025 Broken Access Control

These are mappings of the inspected FRX paths, not severity scores. Read the full requirement and CWE definition before using an ID. Record the ASVS version and selected review scope.

Resolve a wording conflict

Checklist: “Does the centralized validation check block all the special characters?”
— PDF p. 2.

For `/search`, consider an ordinary title containing an apostrophe.

1. Preserve the question's wording and record the behavior you observe.
2. Explain the technical objective: valid search text stays data in the query.
3. Cite **ASVS v5.0.0-1.2.4** and propose a specific interpretation or deviation.
4. Name the requirement owner who must decide; keep that decision pending.

The reviewer can recommend parameter binding and suitable input rules. The reviewer cannot silently waive a binding local requirement.

Decide applicability with a scope reason

Question	Work needed before disposition
XML schema validation (PDF p. 4)	Inspect routes, parsers, and accepted formats. If XML is outside this build, document the bounded N/A reason.
Audit-log review process (PDF p. 2)	Request a procedure and evidence of review. Missing evidence leaves an open question.
Session inactivity timeout (PDF p. 4)	Inspect configuration and how activity affects expiration; define later timing cases.

An absent control on an applicable path may support a **Fail**. An unexamined path or unavailable operating record needs follow-up. Do not use N/A to close either one.

Practicum 3 • build six evidence records

Work in pairs using the **vulnerable state**, model, source, and questionnaire. Reuse your existing request traces.

1. **5 min:** rank your first two review questions by consequence and reachability.
2. **20 min:** write six records: one each for A, C, D, and E; two for B (access and session). Include source or a specific evidence request.
3. **5 min:** add one version-qualified ASVS criterion and justified CWE mapping; record one applicability or wording decision.
4. **5 min:** exchange records and challenge one unsupported conclusion.

Submit: the six records, review order, and follow-up list. Use **practicum-3-checklist-worksheet.md**.

Suggested 35-minute learner activity, adjustable to schedule. Source review only: no source repair, service startup, reset, or payload execution required. Do not use remediated code as the assessed state.

Peer check: could another reviewer repeat it?

- Can you locate the exact questionnaire item and the cited code?
- Does the conclusion match the whole path, including rejection and error branches?
- Is a proposed control being mistaken for an existing control?
- Does a Pass claim cover the full question, including words such as “all”?
- Is N/A supported by scope? Are unknowns assigned to an evidence owner?
- Does the mapping describe the root cause, and is the criterion version recorded?

Return **one concrete correction** and **one next verification case**. Keep static observations separate from runtime results.

Carry one record into the risk assessment

Use the supplied **APPROVE** structure to prepare one draft risk entry.

- **Threat event:** actor, precondition, action, and affected asset.
- **Vulnerability / predisposing characteristics:** observed behavior and evidence.
- **Mitigating factors:** verified limits, with assumptions identified separately.
- **Security references:** checklist locator, requirement, and relevant ASVS/CWE.
- **Likelihood, impact, risk:** explain the basis; identify unknowns and use the template's reference tables.

Assign the next action and evidence owner. Keep recommendations, deviations, and acceptance decisions visibly separate; students do not grant approval.

Practicum 3 resources and hand-off

- **Questionnaire:** [Code_Review_Checklist_Fillable.pdf](#), PDF pp. 1–4 / printed pp. 196–199. Use page + exact question wording.
- **Evidence:** vulnerable FRX source, threat model, prior traces, and [models/requirements-and-architecture.md](#).
- **Work product:** [practicum-3-checklist-worksheet.md](#); keep the supplied PDF and APPROVE template unchanged.
- **Verification criteria:** [ASVS 5.0.0](#), especially 1.2.4 and 8.2.2.
- **Classification:** [OWASP Top 10:2025](#), [CWE-89](#), and [CWE-862](#).

Next: Practicum 4 validates manual observations and tool/supply-chain signals. Bring open cases, evidence requests, and the exact source revision.

DAY 2 • PRACTICUM 4

Validate scanner and supply-chain evidence

Inspect a Python scanner result, follow transitive dependencies, and use an SBOM to describe the assessed environment.

Keep the tool's claim separate from your conclusion.

Four artifacts answer different questions

Artifact	Question it helps answer	What still needs review
Static-analysis result	Does source match a risky pattern?	Reachability, controls, and actual consequence
Dependency audit	Is a package/version linked to an advisory?	Identity match, affected feature, and applicability
SBOM	Which components and relationships were inventoried?	Completeness and the exact build/environment
Security test	Does this case meet the expected behavior?	Alternate paths and untested conditions

Use all four with your manual evidence. A quiet scanner or passing test suite does not establish complete security.

Run a Python static analyzer

Bandit inspects Python syntax for security patterns without serving FRX requests. Use the prepared tool environment; setup is in [practicums-4-5-lab.md](#).

```
$src = "frx-project/src/frx_portal"  
& $toolPy -m bandit -r $src -f json -o evidence/bandit.json  
$LASTEXITCODE
```

- Keep tool version, command, source hash, scan scope, exclusions, and errors.
- Review both severity and confidence. They describe the tool's report.
- A findings exit code is different from a scan that did not complete.

Actual FRX scanner output: inspect the claim

Selected JSON fields from the Bandit run; file path shown separately:

`frx-project/src/frx_portal/states/vulnerable.py`

```
{
  "line_number": 107,
  "test_id": "B608",
  "issue_severity": "MEDIUM",
  "issue_confidence": "LOW",
  "issue_text": "Possible SQL injection vector through string-based query construction."
}
```

At `/search`, trace `q` into the query and then `connection.execute`. The source evidence can be stronger than the scanner's confidence label.

Exact selected fields: `instructor/evidence/bandit-sample.json`; full report: `bandit-frx.json`. Source: `states/vulnerable.py:102-110`. Run metadata and source hashes accompany the report.

Triage findings against the whole path

Reported signal	What the FRX reviewer must check
B608 at vulnerable line 94	The route converts the report ID to an integer. This string is not the same injection path as free-form <code>/search</code> . Review authorization separately.
B310 at remediated line 223	Inspect the configured origin and constructed path. A <code>urlopen</code> warning does not establish unrestricted outbound access.
B403 at vulnerable line 6	<code>pickletools</code> inspects opcodes here; the route does not unpickle or execute them.
B324 at vulnerable lines 30/58	MD5 is used for passwords. A tool hint such as <code>usedforsecurity=False</code> would not correct password storage.

Record: **supported, not supported for this path, or needs evidence**. Explain why; do not suppress a rule just to obtain a clean report.

Direct dependencies bring other packages

Selected relationships from the installed FRX environment:

```
FRX
├─ Flask 3.1.2          direct runtime dependency
│   └─ Jinja2 3.1.6     transitive to FRX
│       └─ MarkupSafe 3.0.3
│           └─ Werkzeug 3.1.8    transitive to FRX
└─ cryptography 46.0.5    direct runtime dependency
    └─ cffi 2.1.1 → pycparser 3.0
```

This is a **partial graph**. Flask also depends directly on MarkupSafe and other packages.

pytest is a direct **development extra**, with its own dependencies.

Compare declarations with the environment

Source	What it establishes
<code>frx-project/pyproject.toml</code>	Direct runtime requirements and optional development requirements
Root <code>requirements.txt</code>	A pinned package list; it does not explain dependency edges
<code>python -m pip inspect</code>	Installed distribution versions, metadata, and requirements
Environment SBOM	A shareable inventory with component identifiers and relationships

Check differences and their cause. Separate runtime, test, packaging, and scanner environments. The supplied SBOM includes development packages; it is not a production inventory.

Supplied manifests and `instructor/evidence/pip-inspect.json`. Run pip commands with the assessed application's interpreter; install scanners separately so their packages do not pollute that inventory.

OWASP Dependency-Check: use it with context

After installing the official CLI and its required Java runtime, run from the repository root:

```
dependency-check.bat --project FRX --scan requirements.txt `
  --enableExperimental --format HTML --format JSON `
  --out evidence/dependency-check
```

The Python Pip analyzer is **experimental** and recognizes files named `requirements.txt`. It collects package evidence for vulnerability matching.

Refresh the vulnerability data before class; initial updates can be slow. Check analyzer/update errors and inspect package/CPE/version matches. Record justified suppressions with scope, owner, and review date.

Audit the pinned Python package list

```
& $toolPy -m pip_audit -r requirements.txt --no-deps `
--disable-pip -f json -o evidence/pip-audit.json
```

This command audits **only the listed versions**. It does not resolve missing transitives; reconcile the list with the actual environment first.

Our saved run returned **15 advisory records across 3 packages**, containing **8 distinct IDs before alias reconciliation**. Repeated IDs are present. These are tool reports, not 15 confirmed exploitable flaws.

For one record: verify package/version, advisory aliases, affected feature, runtime or dev use, fixed-version claim, and the next check. Record the feed/run date; results change.

Continue checking after today's scan

Tool	Useful role
pip-audit / OSV-Scanner	Repeat package-advisory checks in development or CI; verify supported input and coverage.
OWASP Dependency-Check	Dependency vulnerability reports as part of a build/review process.
Dependabot alerts	Alerts from GitHub's dependency graph when supported dependencies match advisories.
OWASP Dependency-Track	Ingest CycloneDX SBOMs and monitor component risk over time.

Assign an alert owner, review cadence, and response target. Recheck after a dependency change or new advisory. Validate updates with compatibility and security tests.

OSV-Scanner; Dependabot; Dependency-Track. These services were not configured or given repository/SBOM uploads.

An SBOM describes what is in the software

Software Bill of Materials: a machine-readable inventory tied to a particular artifact or environment.

- Identifies components, versions, and relationships between them.
- Can include package identifiers, hashes, suppliers, and license information.
- Helps answer “Do we use this affected component?” when an advisory appears.
- Supports comparisons between builds and records of what was assessed.

Common standards include **CycloneDX** and **SPDX**. An SBOM does not itself prove completeness, absence of vulnerabilities, or that a license obligation has been met.

Generate an SBOM of the intended environment

Use **CycloneDX Python** from the tool environment, targeting the application interpreter:

```
& $toolPy -m cyclonedx_py environment $appPy `
  --pyproject frx-project/pyproject.toml `
  --output-format JSON --output-file evidence/frx.cdx.json
```

Our validated CycloneDX 1.6 output inventories **20 components** from the development environment. Inspect its root component and dependency graph.

For a container or filesystem inventory, **Syft** is another generator supporting CycloneDX and SPDX. Choose a tool and input that match the artifact you need to assess.

Read a component and follow its parent

Selected fields from the generated SBOM:

```
{  
  "bom-ref": "Jinja2==3.1.6",  
  "name": "Jinja2",  
  "version": "3.1.6",  
  "purl": "pkg:pypi/jinja2@3.1.6"  
}
```

bom-ref links the component to graph entries. A **Package URL (purl)** identifies the package ecosystem, name, and version.

Find **Flask==3.1.2** in **dependencies**, then follow its **dependsOn** list to Jinja2. Continue to MarkupSafe. A flat list alone cannot show this path.

Exact selected component fields and graph references: [instructor/evidence/frx-environment.cdx.json](#). The example omits other component metadata.

Check the SBOM before relying on it

1. **Subject:** does it describe the reviewed source/build and environment?
2. **Coverage:** are direct and transitive packages present? Which OS/native components, optional extras, and dev tools are outside or inside scope?
3. **Identity:** do version and purl match the installed package and advisory?
4. **License evidence:** is the value declared metadata, detected text, or missing?
Record unresolved evidence for review.
5. **Freshness:** did the build or dependency set change after generation?

Preserve the inventory and its provenance. Re-audit the same inventory when advisory data changes; regenerate it when the software changes.

Practicum 4 • validate three signals

Work in pairs with the saved reports or repeat the documented commands.

1. **8 min:** triage two Bandit findings. Cite source lines and a reason for each disposition.
2. **7 min:** follow one transitive package to its parent. Compare the manifest, installed metadata, and SBOM.
3. **7 min:** investigate one dependency advisory record. Check duplicates/aliases and separate applicability from the tool's match.
4. **3 min:** record one inventory/license gap and assign follow-up work.

Submit: two source triage notes, one dependency/advisory record, and an SBOM coverage note. Use `practicums-4-5-lab.md`.

Practicum 4 references and saved evidence

- [Bandit CLI](#) and [B608 rule](#).
- [OWASP Dependency-Check Pip analyzer](#), [CLI](#), and [pip-audit](#).
- [CycloneDX Python](#), [CycloneDX SBOM](#), [SPDX](#), and [Syft](#).
- Saved JSON reports, SBOM, package metadata, commands, and source hashes:
[slides/instructor/evidence/](#).

Keep full scanner reports with instructor materials: this run includes remediated source. Record which tools ran and which are documented options only.

DAY 2 • PRACTICUM 5

Security tests: prove the intended behavior

Turn a review finding into an executable check. Run the same expectation against vulnerable and remediated FRX.

A failure should explain which security rule was broken.

pytest: arrange, act, assert

pytest discovers Python test functions, runs them, and reports failed assertions with the observed values.

Part	FRX example
Arrange	Create fresh temporary data and log in as a synthetic partner.
Act	Request another partner's report.
Assert	Expect 403 and no report body.
Fixture	Reusable setup supplied through a named function argument.

conftest.py provides shared fixtures. **tmp_path** supplies a temporary directory. Parameterization runs one test with several inputs and expected outcomes.

A passing test can confirm a vulnerability

FRX's existing **characterization tests** assert the intended contrast:

- Vulnerable report read returns another partner's record.
- Remediated report read rejects that same access.
- The test passes when both states behave as the fixture expects.

A **security acceptance test** asserts the required protection in either state. The cross-report check should fail on vulnerable FRX and pass on remediated FRX.

Keep the security expectation constant. Do not change 403 to 200, add an **xfail**, or skip the test to make a vulnerable result look successful.

Prepare the isolated test environment

From the repository root, using Python 3.12–3.14:

```
$appPy = (Resolve-Path .venv/Scripts/python.exe).Path  
& $appPy -m pip install -e "./frx-project[dev]"  
$tests = "slides/instructor/security_tests"  
& $appPy -m pytest $tests --collect-only -q
```

The instructor reference collects **six cases**. Create a virtual environment first if needed; full setup is in the lab handout.

The harness puts all app files under pytest's temporary directory. Flask's test client sends requests in-process; no portal server or **frx-reset** command is needed.

A fixture makes the test repeatable

Core setup from the **lab** fixture; **state** comes from **--frx-state**:

```
settings = Settings(state=state, runtime_root=tmp_path / "runtime")
app = create_app(state, settings)
app.config.update(TESTING=True)
client = app.test_client()
```

The **signed_in** fixture logs in as **partner.alex**, asserts a successful login, and returns **(client, settings)**.

Use **base_url="https://localhost"** consistently for login and protected requests so the remediated Secure cookie is sent. This simulates HTTPS request context; it does not test real TLS or certificates.

Test allowed and denied report access

```
@pytest.mark.parametrize("report_id, expected", [(101, 200), (102, 403)])
def test_partner_report_access(signed_in, report_id, expected):
    client, _ = signed_in
    response = client.get(f"/reports/{report_id}", base_url="https://localhost")
    assert response.status_code == expected
    if expected == 200:
        assert response.get_json()["report"]["id"] == report_id
    else:
        assert "report" not in response.get_json()
```

Alex owns 101, not 102. The positive case prevents “deny everything” from passing. The denied case checks both status and absence of report data.

Exact test: `instructor/security_tests/test_security_acceptance.py:8-16`. Ownership: `frx-project/src/frx_portal/database.py:15-36`; expected access: model REQ-01. Import pytest in the test file.

Confirm a login injection is rejected

```
def test_login_rejects_sql_as_credentials(lab):
    client, _ = lab
    response = client.post(
        "/login",
        json={"username": "nobody' OR 1=1 --", "password": "wrong"},
        base_url="https://localhost",
    )
    assert response.status_code == 401
    assert client.get("/reports/101", base_url="https://localhost").status_code == 401
```

Use a fresh client. Confirm that rejection also leaves protected data inaccessible. The ordinary login in `signed_in` remains a separate positive control.

Check encoding with harmless markup

```
def test_greeting_keeps_markup_as_text(lab):  
    client, _ = lab  
    response = client.get("/welcome", query_string={"name": "<b>field</b>"})  
    assert response.status_code == 200  
    assert "<b>field</b>" not in response.text  
    assert "&lt;b&gt;field&lt;/b&gt;" in response.text
```

The two body assertions detect both unescaped markup and accidental removal of the submitted text. This checks the HTML response; the Flask client does not execute JavaScript or prove browser-wide XSS protection.

Assert side effects as well as responses

Finding	Arrange and act	Required assertions
Password appears in audit log	Successful synthetic login; read only the temporary test log	Password absent; expected success event present
Invalid PDF accepted	Authenticated upload of not a PDF as evidence.pdf	Status 400; upload-directory entries unchanged

The supplied tests implement both cases. Extend them with failed login and valid file intake as additional checks.

Do not claim complete PDF validation from a header check. A successful rejection case also says nothing about file quotas, retention, or later retrieval.

Run the same assertions against both states

```
& $appPy -m pytest $tests --frx-state vulnerable -q --tb=short  
& $appPy -m pytest $tests --frx-state remediated -q --tb=short
```

Observed instructor results:

State	Result	Meaning
Vulnerable	5 failed, 1 passed	Five security expectations violated; own-report control passed
Remediated	6 passed	These six cases met their expected behavior

Read the failing assertion: `assert 200 == 403` shows the request reached the route but access was not denied. Import errors or failed login setup do not confirm the intended finding.

Practicum 5 • extend the security tests

Write your test before comparing it with the instructor reference.

1. **5 min:** select a finding and state the security rule, input, identity, and expected result.
2. **10 min:** add one positive control and one rejection case. Check response data or a side effect as appropriate.
3. **5 min:** run the unchanged assertions against both states. Explain each failure before treating it as evidence.
4. **5 min:** add an alternate-path case and record remaining gaps.

Suggested extension: log in as Alex and search for **Watershed**. Expect no other partner's report; confirm a **Prairie** search still returns Alex's report.

Submit test code, commands, state/revision, actual output, linked finding, and coverage limits. Start: states/vulnerable.py:102–110; states/remediated.py:147–160; database.py:22–45. Preserve the supplied teaching states.

Close the finding with bounded evidence

- Link the test to the finding, requirement, and exact corrected code.
- Preserve the failing vulnerable result and passing remediated result.
- Run adjacent tests and the existing suite; investigate unexpected changes.
- State which inputs, roles, alternate routes, errors, and side effects remain untested.
- Update the checklist record and APPROVE risk entry with verified controls and remaining risk.

For example, a passing `/reports/102` denial test does not establish that `/search` enforces the same policy. Likewise, a clean dependency scan does not prove the deployed environment matches the scanned inventory.

Carry forward Practicum 3 records and APPROVE's mitigating factors/security references. Test results support a recommendation; they do not grant risk acceptance or establish production readiness.

Practicum 5 resources

- [pytest fixtures](#), [parameterization](#), and [test invocation](#).
- [Flask test client and fixtures](#).
- Setup, commands, exercises, and result interpretation: [slides/practicums-4-5-lab.md](#).
- Instructor reference: [slides/instructor/security_tests/](#); expected-state test results and provenance: [slides/instructor/evidence/](#).
- Existing characterization and regression tests: [frx-project/tests/](#).

Keep complete reference tests and remediated code out of learner packages that withhold solutions. Use the vulnerable source and learner prompts first, then release the reference for comparison.

Day 1: decide what needs protection

- **Set the scope:** mission, stakeholders, assets, identities, and assumptions.
- **Model the system:** data flows, stores, privileges, and trust boundaries.
- **Describe threats:** actor, precondition, action, asset, and consequence; use STRIDE and abuse cases.
- **Prioritize and respond:** weigh likelihood, impact, and uncertainty; assign an action and owner.
- **Make it verifiable:** write requirements, expected evidence, residual risk, and change triggers. Get a peer review.

FRX example: a partner changes a report ID. The model asks who should be allowed to read the selected record.

Day 2: check the implementation and evidence

- Turn prioritized threats into a scoped review plan.
- Trace requests through Python code, identity, checks, outputs, and side effects.
- Answer applicable questionnaire items with evidence and explicit gaps.
- Investigate scanner claims; inspect direct/transitive packages and SBOM coverage.
- Test the required behavior, compare states, and record what remains untested.

FRX example: trace report access, test Alex's access to reports 101 and 102, then inspect `/search` as another path to the data.

Keep the chain visible: **threat → requirement → code → evidence → recommendation.**

Challenges for analysts and ISSEs

Challenge	Practical response
Too little review time	Prioritize consequential paths; record omitted work and its owner.
Vague or conflicting requirements	Preserve the wording; ask its owner to resolve the interpretation.
Missing operating evidence	Identify unverified deployment, identity, logging, and recovery claims.
Conflicting or quiet tools	Trace the behavior, check coverage, and test the disputed claim.
Changing software and threats	Record the assessed build, recheck triggers, and monitoring owner.

Make uncertainty visible. Unexplained suppressions and assumed controls can mislead the decision-maker.

Give a recommendation someone can act on

State **what you reviewed, what the evidence supports, what remains open, and what must happen next.**

The six acceptance cases passed against the supplied remediated FRX state. This supports the tested behaviors in the local fixture. Other paths and deployment controls still need evidence; these results do not establish readiness for operational use.

Then name the proposed action, owner, required proof, and decision date. Carry those conditions and residual risks into the checklist notes and APPROVE assessment.

Revisit the recommendation when code, dependencies, configuration, exposure, requirements, or relevant advisory information changes.

Illustrative delegate brief grounded in instructor/evidence/security-remediated.xml and model ASM-01. No operational approval or risk acceptance is asserted. APPROVE Template.docx: Scope, Risk Assessment Results, Recommendation.



THREAT MODELING AND SECURE CODE REVIEW

Thank You

Please complete the end-of-course evaluation.